HTML & CSS: A Beginner's Guide to Web Development CSS

Learn to Build Beautiful Websites

Introduction to HTML

1. What is HTML?

HTML stands for **HyperText Markup Language**. It is the standard language used to create and structure content on the web. HTML forms the backbone of most web pages and tells web browsers how to display text, images, links, and other content.

HTML is not a programming language. It is a markup language used to create the structure of web pages.

History of HTML – Who Created It?

Year	Event
1989	Tim Berners-Lee, a British computer scientist at CERN, proposed a system to share documents over the internet.
1991	He created the first version of HTML with just 18 tags.
1995	HTML 2.0 was officially published – it standardized the basic structure.
1997-1999	HTML 3.2 and HTML 4.01 introduced more features like forms, tables, and scripting support.
2008	HTML5 was introduced by W3C and WHATWG – supporting modern multimedia, semantic elements, and better forms.
2014	HTML5 became the official standard. It is the current version in use today.

2. Structure of an HTML Document

Every HTML file follows a specific structure. Here's a sample:

```
<!DOCTYPE html>
<html>
<head>
<title>Techpile Technology</title>
</head>
<body>
<h1>Summer Tranning</h1>
<p>Apprenticeship Program</p>
</body>
</html>
```

Explanation:

Tag	Purpose
<!DOCTYPE html>	Tells the browser to use HTML5
<html>	Root element of the web page
<head>	Contains metadata (title, charset, links to CSS, etc.)
<title>	The name shown on the browser tab
<body>	All visible content goes here (text, images, buttons, etc.)

3. Tags and Elements

Tags:

Tags are keywords enclosed in angle brackets < >.

Techpile Technology
 And Code Helper

Elements:

An element is the full structure, including opening tag, content, and closing tag:

Techpile Technology. ← Techpile Technology
Some tags are self-closing (they don't need a closing tag):
<img src="logo.png" alt="Logo">

<hr>

4. Attributes

Attributes provide extra information about HTML elements. They go inside the opening tag.

Example:

<img src="image.jpg" alt="Description" width="300">
Visit

Common Attributes:

- src: Source of an image
- alt: Alternate text for image
- href: Hyperlink destination

- target="_blank": Opens link in a new tab
- id / class: Used in CSS/JavaScript for styling and functionality

5. Basic HTML Code

Here's a full HTML5 boilerplate template you can use in any project:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Code Helper </title>
</head>
<body>
<h1>Welcome To Techpile Technology </h1>
<p>And Code Helper>
</body>
</html>
```

Important Tags:

- <meta charset="UTF-8">: Supports all character sets
- <meta name="viewport">: Makes the layout responsive for mobile devices
- <title>: Sets the page title
- <body>: Contains everything that shows up in the browser

Summary

Concept	What it Means
HTML	Language used to create web page structure
Created by	Tim Berners-Lee in 1989
Current version	HTML5
Tags	Mark sections of content
Attributes	Give extra meaning to tags (like src, href)
Structure	HTML pages have <html>, <head>, <body>, etc.

6.Important Tags in the <head> Section of HTML

1. <meta charset="UTF-8">

Sets the character encoding of the web page (how text is stored).

Why use it?

To support all standard characters (like English, Hindi, emojis, etc.)

Example:

<meta charset="UTF-8">

2. <meta name="viewport" content="width=device-width, initial-scale=1.0">

Makes the website responsive on all screen sizes, especially mobile.

Why use it?

Without this tag, your site might look zoomed out or broken on phones.

Example:

<meta name="viewport" content="width=device-width, initial-scale=1.0">

3. <title>

Sets the title of the web page shown in the browser tab or in search results.

Why use it?

Very important for SEO and branding.

Example:

<title>Techpile Technology</title>

4. <link rel="stylesheet" href="style.css">

Links an external CSS file to style the webpage.

Why use it?

Keeps HTML and CSS separate, cleaner, and easier to manage.

Example:

<link rel="stylesheet" href="style.css">

5. <script src="script.js" defer></script>

Links an external JavaScript file for interactivity.

Why use it?

To make the webpage dynamic (e.g., animations, form validation, etc.)

Example:

<script src="script.js" defer></script>

6. <meta name="description" content="...">

Gives a brief description of the web page for search engines.

Why use it?

Improves SEO and affects how your site appears in Google results.

Example:

<meta name="description" content="This is a personal blog about travel and coding.">

7. <meta name="keywords" content="HTML, CSS, JavaScript">

Provides keywords about the content of your webpage.

Why use it?

Helps search engines (though less important today for SEO).

Example:

<meta name="keywords" content="web design, html5, css3, javascript">

8. <meta name="author" content="Your Name">

Tells who is the author of the web page.

Why use it?

Good for credit, documentation, and sometimes for SEO.

Example:

<meta name="author" content="John Doe">

Final Example with All 8 Tags:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Learn HTML in a simple way with examples.">
  <meta name="keywords" content="HTML, CSS, JavaScript, Web Development">
  <meta name="author" content="John Doe">
  <title>HTML Tutorial</title>
  <link rel="stylesheet" href="style.css"><script src="script.js" defer></script></head>
```

Module 2: Text Formatting Tags in HTML

1. <h1> to <h6> — Headings

HTML provides 6 levels of headings. They define the importance of the content — <h1> is the most important, and <h6> the least.

<h1>HTML & CSS</h1>

<h2>JavaScript</h2>

<h3>JQuery</h3>

<h4>DataBase Management System</h4>

<h5>BootStrap </h5>

<h6>C Programming </h6>

Use cases:

- <h1> is usually the main page title.
- Lower headings (<h2>, <h3>) are used for subheadings.

2. — Paragraph

Used to define a block of text.

Techpile Technology.

Browsers automatically add space before and after a paragraph.

**3.
 and <hr>**

**
 — Line Break**

Inserts a single line break (like pressing Enter once).

 Techpile Technology.
And Code Helper.

<hr> — Horizontal Rule

Draws a horizontal line across the page. Often used to separate sections.

Summer Tranning

<hr>

 Apprenticeship Program

**Both
 and <hr> are self-closing tags.**

4. Text Styling Tags

** and **

- : Makes text bold (for visual emphasis).
- : Makes text bold + has semantic meaning (important text).

This is bold text.

This is very important text.

 and

- : Italics for styling.
- : Italics with emphasis (semantic importance).

This is italic text.

This is emphasized text.

<u> — Underline

This is <u>underlined</u> text.

5. <blockquote>, <pre>, <code>

<blockquote> — Quoted Text

Used for long quotations. It usually gets indented by the browser.

<blockquote>

"The only limit to our realization of tomorrow is our doubts of today."

</blockquote>

<pre> — Preformatted Text

Keeps spaces, tabs, and line breaks as typed.

```
<pre>
Line 1
    Line 2 (indented)
Line 3
</pre>
```

<code> — Inline Code

Used to show short pieces of code (inline).

```
<p>Use the <code>print()</code> function in Python.</p>
```

Combine <pre> and <code> to display multi-line code blocks:

```
<pre><code>
def hello():
    print("Hello, world!")
</code></pre>
```

6. Comments in HTML

Used to leave notes in your code. Comments are ignored by the browser.

```
<!-- This is a comment -->
<p>This is visible content</p>
<!-- <p>This won't be shown on the page</p> -->
```

Summary Table

Tag	Description
<h1> to <h6>	Headings (most to least important)
<p>	Paragraph
 	Line break
<hr>	Horizontal line
<b>	Bold (visual only)

Tag	Description
	Bold (semantic importance)
<i>	Italics (visual)
	Italics (emphasized meaning)
<u>	Underlined text
<blockquote>	Quoted block
<pre>	Preserves whitespace
<code>	Displays code
<!-- comment -->	Adds comments in code

Mini Practice Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Code Helper</title>
</head>
<body>
  <h1> Apprenticeship Program </h1>
  <h2>Summer Tranning</h2>
  <p>This is a <strong>very important</strong> paragraph with a <br>line break.</p>
  <hr>
  <p>Code: <code>console.log("Hello");</code></p>
  <!-- This is a comment -->
</body>
</html>
```

Module 3: Links and Images

1. Anchor Tag — <a>

The <a> tag stands for **Anchor**. It is used to create a **link** to:

- another webpage
- another section on the same page
- email address
- phone number

Why is it used?

To let users **navigate** or **take action** (like visiting a page, sending an email, etc.).

Syntax:

Link Text

Attribute	Description
href	The destination URL
target	Controls where the link opens (_blank = new tab)

Example:

Open Google

Output:

A clickable link:
Open Google (opens in a new tab)

2. Image Tag — <img>

The <img> tag is used to **display images** on a web page. It is a **self-closing tag** (doesn't need a closing tag).

Why is it used?

To add **visual content** like logos, photos, illustrations, etc., to your website.

Syntax:

<img src="image-path" alt="text" width="value" height="value" title="tooltip">

Attribute	Description
src	Source (file path or URL)
alt	Alternative text if image fails to load
width	Width of the image (in px or %)
height	Height of the image
title	Tooltip shown on hover

Example:

<img src="images/dog.jpg" alt="A cute dog" width="300" height="200" title="Puppy">

Output:

Shows an image of a dog, 300px wide and 200px tall.
When hovered, it shows a tooltip: **"Puppy"**
If the image is missing, it shows text: **"A cute dog"**

Let's Summarize Both:

Tag	Purpose	Syntax	Common Attributes
<a>	Creates links	Link	href, target
<img>	Shows images	<img src="">	src, alt, width, height, title

Bonus Tip — Image Path Types:

Type	Example	Meaning
Relative Path	img/pic.jpg	Image is inside an "img" folder in same project
Absolute Path	https://example.com/pic.jpg	Image from the internet
Parent Folder	../pic.jpg	One level up from current folder

Module 4: Lists in HTML

What are Lists in HTML?

Lists are used to group related items in a structured format on a web page.

There are **three types** of lists:

Type	Used For
Ordered List ( )	Numbered list (1, 2, 3...)
Unordered List ( )	Bullet point list
Description List (<dl>)	Term-description list (like a glossary or FAQ)

**1. Ordered List: <ol> + **

What is it?

An **ordered list** displays items in a **numbered sequence**.

Why is it used?

Use it when the **order matters**, like steps in a recipe or ranking.

Syntax:

```
<ol>
  <li>Diploma</li>
  <li>B.Tech</li>
</ol>
```

- = ordered list (container)
- = list item

Example:

```
<ol>
  <li>M.Tech</li>
  <li>BCA</li>
  <li>MCA</li>
</ol>
```

Output:

1. M.Tech
2. BCA
3. MCA

**2. Unordered List: <ul> + **

An **unordered list** shows items with **bullet points**.

Why is it used?

Use it when the **order doesn't matter**, like a list of groceries or hobbies.

Syntax:

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
```

Example:

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>C Programming</li>
</ul>
```

Output:

- HTML
- CSS
- C Progranning

3. Description List: <dl>, <dt>, <dd>

A **description list** is used to display **terms and their descriptions**.

Why is it used?

Use it for **glossaries**, **FAQs**, or **product specifications**.

Syntax:

```
<dl>
  <dt>Term</dt>
  <dd>Description</dd>
</dl>
```

- <dl> = description list (container)
- <dt> = description term
- <dd> = description details

Example:

```
<dl>
  <dt>HTML</dt>
  <dd>A markup language for creating web pages.</dd>
  <dt>CSS</dt>
  <dd>Stylesheet language used to design web pages.</dd>
</dl>
```

Output:

HTML
A markup language for creating web pages.

CSS
Stylesheet language used to design web pages.

Summary Table

List Type	Tag	Description	When to Use
Ordered List	+	Numbered items	Steps, instructions
Unordered List	+	Bulleted items	Non-sequential items
Description List	<dl> + <dt> + <dd>	Terms & definitions	Glossaries, FAQs

Full Practice Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Techpile technology</title>
</head>
<body>
  <h2>Ordered List (To-Do)</h2>
  <ol>
```

```
    <li>Study HTML</li>
    <li>Practice Coding</li>
    <li>Build a Website</li>
  </ol>
  <h2>Unordered List (Shopping List)</h2>
  <ul>
    <li>Milk</li>
    <li>Bread</li>
    <li>Eggs</li>
  </ul>
  <h2>Description List (Glossary)</h2>
  <dl>
    <dt>HTML</dt>
    <dd>HyperText Markup Language</dd>
    <dt>CSS</dt>
    <dd>Cascading Style Sheets</dd>
  </dl>
</body>
</html>
```

Module 5: Tables

What is an HTML Table?

An HTML table is a way to present **Tabular data** (rows and columns) on a web page—think of a spreadsheet or database output.

Why use tables?

- To display structured information: schedules, pricing grids, comparison charts, etc.
- Semantically meaningful: screen readers and search engines understand the data structure.

Basic Table Syntax

```
<table>
  <caption>Table Caption (optional)</caption>
  <tr>                <!-- Table Row -->
    <th>Header 1</th>    <!-- Table Header Cell -->
    <th>Header 2</th>
  </tr>
  <tr>
    <td>Data 1</td>      <!-- Table Data Cell -->
    <td>Data 2</td>
  </tr>
</table>
```

- <table>: container for the entire table
- <caption>: title or description (appears above the table)
- <tr>: table row
- <th>: header cell (bold, centered by default)
- <td>: standard data cell

Example: Simple Price List

```
<!DOCTYPE html>
<html>
<head>
  <title>Techpile Technology</title>
</head>
<body>
  <table border="1" cellpadding="8" cellspacing="0">
    <caption>Tranning</caption>
    <tr>
      <th>Summer Tranning</th>
      <th> Apprenticeship Program </th>
      <th>  Winter Training  </th>
    </tr>
```

```
    <tr>
      <td>HTML</td>
      <td>Python</td>
      <td>.NET </td>
    </tr>
    <tr>
      <td>CSS</td>
      <td>PHP</td>
      <td>Java</td>
    </tr>
    <tr>
      <td>JavaScript</td>
      <td>Mern Stack</td>
      <td>Python</td>
    </tr>
  </table>
</body>
</html>
```

Colspan & Rowspan

Colspan:

Merges cells **horizontally** across multiple columns.

```
<tr>
  <td colspan="2">Merged across two columns</td>
  <td>Normal Cell</td>
</tr>
```

Rowspan:

Merges cells **vertically** across multiple rows.

```
<tr>
  <td rowspan="2">Merged down two rows</td>
  <td>Row 1, Col 2</td>
</tr>
<tr>
  <!-- first cell is taken by the rowspan above -->
  <td>Row 2, Col 2</td>
</tr>
```

Table Borders, Padding, Spacing

- **border="1"** (deprecated HTML attribute) adds a 1px border around cells.
- **cellpadding="8"** adds 8px **inner padding** inside each cell.
- **cellspacing="0"** removes any **space between cells**.

Modern CSS approach

```
table {
  border-collapse: collapse; /* merges shared borders */
}
th, td {
  border: 1px solid #ccc;
  padding: 8px;
}
<table>
  …
</table>
```

Table Caption

A <caption> provides a **title or summary** for the table.

Why use it?

- Improves **accessibility**: screen readers announce it.
- Gives context to the data.

Syntax & Example:

```
<table>
  <caption>Employee Directory</caption>
  <tr>
<th>Name</th>
<th>Role</th>
</tr>
  <tr>
<td>Alice</td>
<td>Developer</td>
</tr>
  <tr>
<td>Bob</td>
<td>Designer</td>
</tr>
</table>
```

Output: “Employee Directory” displayed above the table, centered by default.

Full Practice Example

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Code Helper</title>
  <style>
    table { border-collapse: collapse; width: 100%; }
    th, td { border: 1px solid #333; padding: 6px; text-align: left; }
    caption { caption-side: top; font-weight: bold; margin-bottom: 8px; }
  </style>
</head>
<body>
  <table>
    <caption>Quarterly Sales Report</caption>
    <tr>
      <th>Quarter</th>
      <th>Sales ($)</th>
      <th>Change</th>
    </tr>
    <tr>
      <td>Q1</td>
      <td>50,000</td>
      <td>—</td>
    </tr>
    <tr>
      <td>Q2</td>
      <td>65,000</td>
      <td>+30%</td>
    </tr>
    <tr>
      <td rowspan="2">Q3 & Q4</td>
      <td>80,000</td>
      <td>+23%</td>
    </tr>
    <tr>
      <td>90,000</td>
      <td>+12.5%</td>
    </tr>
    <tr>
      <td colspan="2">Total Yearly Sales</td>
      <td>285,000</td>
    </tr>
```

```
  </table>
</body>
</html>
```

What you'll see:

- A styled table with borders and padding.
- A caption "Quarterly Sales Report."
- Q3 & Q4 combined into one cell using rowspan="2".
- A final row merging two cells with colspan="2".

Module 6: Forms

1. Basic HTML Form Elements

<form> tag:

- Used to create a form.
- Contains form controls like inputs, buttons, selects, etc.

Attributes:

- action: URL where form data will be sent after submission.
- method: HTTP method used to send data — usually "get" or "post".

SYNTAX:

```
<form action="/submit" method="post">

  <!-- form controls -->

</form>
```

2. Form Controls

<input> tag — most versatile form control.

Common input types:

Type	Purpose
text	Single line text input
password	Password input (masked)
checkbox	Multiple selection
radio	Single selection from options
file	Upload files
submit	Button to submit the form

Example:

```
<input type="text" name="username">

<input type="password" name="password">

<input type="checkbox" name="subscribe" value="yes">
```

<input type="radio" name="gender" value="male">

<input type="file" name="resume">

<input type="submit" value="Submit">

<textarea>:

- For multiline text input.

SYNTAX:

<textarea name="message" rows="4" cols="50" placeholder="Write your message here"></textarea>

<select> and <option>:

- Dropdown menu for selecting one or multiple options.

SYNTAX:

<select name="country">

<option value="us">United States</option>

<option value="uk">United Kingdom</option>

<option value="in">India</option>

</select>

<label>, <fieldset>, <legend>:

- <label>: Provides a clickable label for inputs.

SYNTAX:

<label for="email">Email:</label>

<input type="email" id="email" name="email">

- <fieldset>: Groups related controls inside a box.
- <legend>: Title for the fieldset group.

SYNTAX:

```
<fieldset>
  <legend>Personal Information</legend>
  <input type="text" name="firstname" placeholder="First Name">
  <input type="text" name="lastname" placeholder="Last Name">
</fieldset>
```

4. **Attributes for inputs:**

Attribute	Meaning & Usage
required	User **must fill** this field before submitting the form
placeholder	Shows a **hint** inside the input when it's empty
disabled	Field is **not editable** and **not submitted** with form
readonly	Field is **not editable** but **value will be submitted**

4. Form Validation Attributes

What are form validation attributes?

They are HTML attributes that allow the browser to **check user input automatically** before the form is submitted. This reduces the need for manual validation with JavaScript.

Why use form validation attributes?

- To **ensure data correctness and completeness** before sending to the server.
- To **improve user experience** by showing instant errors.
- To reduce server-side validation errors.
- To simplify coding by leveraging built-in browser functionality.

Features of form validation attributes:

Attribute	What it does
required	Makes a field mandatory
pattern	Defines a regex pattern that the input must match
min, max	Set minimum and maximum values for number/date inputs
minlength,	Set limits on input length

Attribute	What it does
maxlength	
step	Sets the intervals for numbers/dates
type	Different input types trigger specific validation (email, url, number)

Syntax and Examples

1. **required**

SYNTAX:

<input type="text" name="username" required>

User cannot submit the form without filling this field.

2. **placeholder**

SYNTAX:

<input type="email" name="email" placeholder="Enter your email">

Shows hint text inside the input box.

3. **disabled**

SYNTAX:

<input type="text" name="phone" disabled value="Not editable">

Field is greyed out, cannot be edited or submitted.

4. **readonly**

SYNTAX:

<input type="text" name="referral" readonly value="REF12345">

User cannot edit, but value will be sent.

5. **pattern**

SYNTAX:

<input type="text" name="username" pattern="[A-Za-z]{3,}" title="Only letters, min 3 characters" required>

User input must be at least 3 letters (no numbers/special chars).

6. **min, max**

SYNTAX:

<input type="number" name="age" min="18" max="99" required>

Age must be between 18 and 99.

7. **minlength, maxlength**

SYNTAX:

<input type="text" name="password" minlength="6" maxlength="12" required>

Password length between 6 and 12 characters.

8. **step**

SYNTAX:

<input type="number" name="quantity" step="5" min="0">

User can enter values like 0, 5, 10, 15, etc.

5. Complete Form Example with Validation

SYNTAX:

<form action="/submit" method="post">

<fieldset>

<legend>Registration Form</legend>

<label for="username">Username:</label>

<input type="text" id="username" name="username" required pattern="[A-Za-z]{3,}"

title="Username should contain at least 3 letters only">

<label for="email">Email:</label>

```
<input type="email" id="email" name="email" required
placeholder="example@mail.com"><br><br>

<label for="age">Age:</label>

<input type="number" id="age" name="age" min="18" max="99" required><br><br>

<label for="bio">Bio:</label><br>

<textarea id="bio" name="bio" placeholder="Tell us about yourself"
maxlength="200"></textarea><br><br>

<label for="resume">Upload Resume:</label>

<input type="file" id="resume" name="resume"><br><br>

<input type="submit" value="Register">

</fieldset>

</form>
```

Summary:

Topic	**Explanation**
<form>	Container for inputs, submits data
action and method	Where and how data is sent
<input> types	Text, password, checkbox, radio, file, submit
<textarea>	Multi-line input
<select>, <option>	Dropdown menu
<label>, <fieldset>, <legend>	Label and group controls
Validation attributes	required, pattern, min, max, readonly, disabled, etc.

Difference Between GET and POST Methods

Both GET and POST are HTTP methods used in HTML forms to send data to a server — but they are used in different ways and for different purposes.

GET Method

1. **Data sent in URL**
   The form data is added to the URL as query parameters (e.g., example.com/page?name=John).
2. **Visible to everyone**
   Since data appears in the address bar, it's not suitable for passwords or private information.
3. **Limited data size**
   You can only send a small amount of data — usually up to 2048 characters.
4. **Can be bookmarked**
   Because the data is part of the URL, you can bookmark the page and revisit it with the same data.
5. **Can be cached**
   The browser may save and reuse the response.
6. **Used for retrieving data**
   Best for things like search forms, filters, or any form that doesn't change data on the server.

Syntax Example

Using GET Method:

```
<form action="/search" method="get">
  <input type="text" name="query" placeholder="Search...">
  <input type="submit" value="Search">
</form>
```

This form will go to:

```
/search?query=yourinput
```

POST Method

1. **Data sent in request body**
   The data is hidden from the URL and sent inside the HTTP request.

2. **More secure**
   Since the data isn't visible in the address bar, it's better for sensitive information like passwords.

3. **No size limit for most cases**
   You can send large amounts of data including file uploads.

4. **Cannot be bookmarked**
   Since the form data is not in the URL, you can't bookmark the submission.

5. **Cannot be cached easily**
   The browser typically won't store POST form results.

6. **Used for sending or saving data**
   Perfect for registration, login, submitting forms, payments, etc.

Syntax Example

Using POST Method:

```
<form action="/register" method="post">
  <input type="text" name="username" required>
  <input type="password" name="password" required>
  <input type="submit" value="Register">
</form>
```

Data is sent **securely** in the request body — **not visible** in the URL.

Module 7: Semantic HTML5 Tags

What is Semantic HTML?

Semantic HTML means using HTML tags **that clearly describe the meaning** of the content inside them.

Instead of using generic tags like or , you use tags like <header>, <footer>, <article>, etc., which tell **what kind of content** they contain.

Why Semantic Tags Matter?

- **SEO (Search Engine Optimization):**
  Search engines (like Google) understand your page better and rank it higher when semantic tags are used properly.
- **Accessibility:**
  Screen readers (used by visually impaired users) can better navigate and describe the page.
- **Cleaner Code:**
  Easier for developers to read, understand, and maintain.

Common Semantic Tags and Their Uses

1. <header>

- Used for the **top section** of a page or section.
- Often contains logo, navigation links, or page title.

Example:

```
<header>
  <h1>My Blog</h1>
  <nav>...</nav>
</header>
```

2. <footer>

- Defines the **bottom section** of a page or section.
- Often includes copyright, links, contact.

Example:

```
<footer>
```

```
  <p>&copy; 2025 My Website</p>

</footer>
```

3. <nav>

- Contains **navigation links** to other parts of the website.

Example:

```
<nav>

  <ul>

    <li><a href="/">Home</a></li>

    <li><a href="/about">About</a></li>

  </ul>

</nav>
```

4. <section>

- Represents a **group of related content** (like a chapter or topic).
- Can have its own heading.

Example:

```
<section>

  <h2>Latest News</h2>

  <p>Today's top story is...</p>

</section>
```

5. <article>

- Used for **independent content** like a blog post, article, or comment.
- Can stand alone outside the page.

Example:

```
<article>

  <h2> Web Design</h2>

  <p> Use semantic HTML...</p>

</article>
```

6. <aside>

- Contains **related or side content** — like a sidebar, ads, tips.

Example:

```
<aside>
  <h3>Related Articles</h3>
  <ul>
    <li>How to Learn HTML</li>
    <li>CSS Basics</li>
  </ul>
</aside>
```

7. <main>

- Wraps the **main content** of the page.
- There should be only **one <main>** per page.

Example:

```
<main>
  <h1>Welcome to My Website</h1>
  <p>This is the homepage.</p>
</main>
```

Importance of Semantic Tags for SEO & Accessibility

SEO (Search Engine Optimization)

- Helps search engines **understand page structure** better.
- Improves how pages appear in search results.
- Tags like <article>, <header>, <nav> give **context to content**.

Accessibility

- **Screen readers** can jump between sections like <main>, <nav>, or <footer>.
- Provides a better experience for users with disabilities.
- Helps tools that assist people navigate websites more efficiently.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Code Helper</title>
</head>
<body>
  <header>
    <h1>Techpile Technology</h1>
    <nav>
      <a href="/">Home</a>
      <a href="/blog">Blog</a>
    </nav>
  </header>
  <main>
    <section>
      <h2>Welcome To Techpile Technology</h2>
      <p>Code Helper</p>
    </section>
    <article>
      <h2>Latest Blog Post</h2>
      <p>Learn about HTML5 semantic tags...</p>
    </article>
  </main>
  <aside>
    <h3>Related Links</h3>
    <ul>
      <li><a href="#">Code Helper</a></li>
      <li><a href="#">Techpile Technology</a></li>
    </ul>
  </aside>
  <footer>
    <p>&copy; 2025 Techpile Technology</p>
  </footer>
</body>
</html>
```

Module 8: Multimedia in HTML5

1. Embedding Videos: <video> Tag

Purpose:

Used to embed **video files** directly into a webpage without needing plugins.

Common Attributes:

- src: path to the video file
- controls: shows play, pause, volume controls
- autoplay: plays automatically when the page loads
- muted: starts muted
- loop: plays video repeatedly
- poster: image shown before video starts

Example:

```
<video width="640" height="360" controls poster="thumbnail.jpg">
  <source src="video.mp4" type="video/mp4">
  Your browser does not support the video tag.
</video>
```

Supports formats like .mp4, .webm, .ogg

2. Embedding Audio: <audio> Tag

Purpose:

Used to embed **audio/music files** into a webpage.

Common Attributes:

- src: path to the audio file
- controls: displays audio controls
- autoplay: plays automatically
- loop: repeats audio
- muted: starts muted

Example:

```
<audio controls>
  <source src="music.mp3" type="audio/mpeg">
  Your browser does not support the audio tag.
</audio>
```

Supports formats like .mp3, .ogg, .wav

3. Using <iframe>

The <iframe> (inline frame) is used to **embed another webpage or content** (like YouTube videos, Google Maps) within your own page.

Common Attributes:

- src: URL of the embedded page
- width & height: dimensions of the frame
- frameborder: (optional) border around iframe
- allowfullscreen: allows full-screen content (for videos)

Embed YouTube Video Example:

```
<iframe width="560" height="315"
  src="https://www.youtube.com/embed/dQw4w9WgXcQ"
  title="YouTube video"
  frameborder="0"
  allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture"
  allowfullscreen>
</iframe>
```

Embed Google Maps Example:

```
<iframe
  src="https://www.google.com/maps/embed?pb=!1m18..."
  width="600" height="450" style="border:0;"
  allowfullscreen=""
  loading="lazy"
  referrerpolicy="no-referrer-when-downgrade">
</iframe>
```

You can get this code from Google Maps by clicking **"Share" → "Embed a map"**.

Summary:

Tag	Used For	Supports
<video>	Playing videos	MP4, WebM, Ogg
<audio>	Playing audio	MP3, WAV, Ogg

<iframe> Embedding external content YouTube, Maps, other websites

Module 1: Introduction to CSS

What is CSS?

CSS (Cascading Style Sheets) is a stylesheet language used to control the appearance and layout of web pages written in HTML. It defines how elements like text, images, buttons, and layouts should look — including their colors, fonts, sizes, spacing, positioning, and even animations**.**

Why use CSS?

- To make web pages **attractive and user-friendly**
- To **separate content from design**
- To apply **consistent styling** across multiple pages
- To make websites **responsive** (work well on all devices)

CSS Syntax

```
selector {
  property: value;
}
```

Example:

```
p {
  color: blue;
  font-size: 16px;
}
```

- p is the **selector** (applies to <p> tags)
- color and font-size are **properties**
- blue and 16px are **values**

Types of CSS

There are **3 main ways to apply CSS**:

1. Inline CSS

- Written **inside an HTML tag**
- Affects only that single element

Example:

```
<p style="color: red; font-size: 18px;">Hello</p>
```

2. Internal CSS

- Written inside a <style> tag in the <head> section
- Affects the whole page

Example:

```
<head>
  <style>
    h1 {
      color: green;
    }
  </style>
</head>
```

3. External CSS

- Written in a **separate .css file**
- Linked to the HTML file
- Best method for large websites

Example in style.css:

```
body {

  background-color: #f2f2f2;

}
```

Link in HTML:

```
<link rel="stylesheet" href="style.css">
```

Place it inside the <head> tag of your HTML.

Complete Example (HTML + CSS)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CSS Example</title>
```

```
  <link rel="stylesheet" href="style.css"> <!-- External CSS -->
  <style>
   h1 {
     color: darkblue;
   }
  </style> <!-- Internal CSS -->
</head>
<body>
  <h1>Welcome</h1>
  <p style="color: green;">This is inline styled text.</p> <!-- Inline CSS -->
</body>
</html>
```

Summary:

Type	Location	Use Case
Inline CSS	Inside HTML tags	For quick, one-time changes
Internal CSS	In <style> tag	For styling a single page
External CSS	In .css file	For large projects/multiple pages

Module 2: CSS Selectors

1. Element Selector

Selects all elements of a particular HTML tag.

Why use it?

To apply the same style to all elements of a given type.

Syntax:

```
element {
  property: value;
}
```

Example:

```
p {
  color: blue;
  font-size: 16px;
}
```

Affects: All <p> tags.

2. Class Selector

Selects all elements with a specific class.

Why use it?

To apply common styles to multiple elements.

Syntax:

```
.className {
  property: value;
}
```

Example:

```
.box {
  border: 2px solid black;
  padding: 10px;
}
```

HTML:

```
<div class="box">HTML</div>
<div class="box">CSS</div>
```

3. ID Selector

Selects a single element with a specific id.

Why use it?

For unique styling or scripting a specific element.

Syntax:

```
#idName {
  property: value;
}
```

Example:

```
#header {
  background-color: lightgray;
  text-align: center;
}
```

HTML:

```
<div id="header">Techpile Technology</div>
```

4. Grouping Selector

Selects multiple elements and applies the same style.

Why use it?

To avoid writing repeated CSS.

Syntax:

```
selector1, selector2 {
  property: value;
}
```

Example:

```
h1, h2, p {
  font-family: Arial;
  margin-bottom: 10px;
}
```

5. Universal Selector

Selects all elements on the page.

Why use it?

Often used to apply a CSS reset or global styles.

Syntax:

```
* {
  margin: 0;
  padding: 0;
}
```

6. Descendant Selector

What is it?

Selects elements that are inside another element (at any level).

Syntax:

```
parent child {
  property: value;
}
```

Example:

```
div p {
  color: green;
}
```

HTML:

```
<div>
  <p>This will be green</p>
</div>
```

7. Child Selector (>)

Selects direct children only (1 level down).

Syntax:

```
parent > child
{
  property: value;
}
```

Example:

```
ul > li {
  color: red;
}
```

HTML:

```
<ul>
 <li>Direct child</li>       <!-- Red -->
 <div><li>Nested</li></div>   <!-- Not affected -->
</ul>
```

8. Attribute Selector

Selects elements based on the presence or value of an attribute.

Syntax:

```
element[attribute="value"] {
 property: value;
}
```

Example:

```
input[type="text"] {
 border: 2px solid blue;
}
```

HTML:

```
<input type="text">
<input type="email">
```

9. Pseudo-classes

Apply styles based on an element's state or position.

Syntax:

```
selector:pseudo-class {
 property: value;
}
```

Common Examples:

:hover

```
button:hover {
  background-color: yellow;
}
```

:focus

```
input:focus {
  border: 2px solid green;
}
```

:nth-child(n)

```
li:nth-child(odd) {
  background: #eee;
}
```

10. Pseudo-elements

Style parts of an element, like the first letter or add content before/after.

Syntax:

```
selector::pseudo-element {
  property: value;
}
```

Examples:

::before

```
h1::before {
  content: " ";
}
```

::after

```
h1::after {
 content: " ";
}
```

Full Working Example:

```
<!DOCTYPE html>
<html>
<head>
 <style>
  /* Element */
  h1 {
   color: darkblue;
  }
  /* Class */
  .highlight {
   background-color: yellow;
  }
  /* ID */
  #main {
   padding: 20px;
  }
  /* Grouping */
  h2, p {
   font-style: italic;
  }
```

```
/* Universal */
* {
  box-sizing: border-box;
}
/* Descendant */
div p {
  color: green;
}
/* Child */
ul > li {
  color: red;
}
/* Attribute */
input[type="text"] {
  border: 1px solid gray;
}
/* Pseudo-classes */
a:hover {
  color: orange;
}
input:focus {
  outline: 2px solid blue;
}
li:nth-child(2) {
  font-weight: bold;
```

```
  }
  /* Pseudo-elements */
  h1::before {
    content: " ";
  }
  h1::after {
    content: " ";
  }
 </style>
</head>
<body>
 <h1>Main Heading</h1>
 <h2>Subheading</h2>
 <p class="highlight">This is a highlighted paragraph.</p>
 <div id="main">
  <p>This is inside the main section.</p>
 </div>
 <ul>
  <li>HTML</li>
  <li>CSS (Bold)</li>
  <li>JavaScript</li>
 </ul>
 <input type="text" placeholder="Enter your name">
 <br><br>
 <a href="#">Code Helper</a></body></html>
```

Module 3: Colors and Backgrounds

Colors in CSS
CSS allows you to define colors in multiple formats:

1. Color Names
CSS supports about 140 **color keywords**.

Example:

color: red;

color: blue;

color: green;

2. HEX Values
A 6-digit code where:

- First 2 digits: Red
- Next 2: Green
- Last 2: Blue

Example:

color: #ff0000; /* red */

color: #00ff00; /* green */

color: #0000ff; /* blue */

You can also use 3-digit shorthand:

Example:

color: #f00; /* same as #ff0000 */

3. RGB
Stands for Red, Green, Blue (values 0–255)

Example:

color: rgb(255, 0, 0); /* red */

4. RGBA
Like RGB, but with Alpha (transparency, 0–1)

Example:

color: rgba(255, 0, 0, 0.5); /* semi-transparent red */

5. HSL
Hue (0–360), Saturation (%), Lightness (%)

Example:

color: hsl(0, 100%, 50%); /* red */

6. HSLA
Same as HSL, but with Alpha

Example:

color: hsla(0, 100%, 50%, 0.5);

Background Properties

1. background-color
Sets the background color of an element.

Example:

background-color: lightblue;

2. background-image
Applies an image as the background.

Example:

background-image: url("image.jpg");

3. background-repeat
Controls if/how the image repeats.

Example:

background-repeat: repeat; /* default */

background-repeat: no-repeat; /* image shows once */

background-repeat: repeat-x; /* horizontal only */

background-repeat: repeat-y; /* vertical only */

4. background-position

Sets the starting position of the background image.

Example:

background-position: top left;

background-position: center center;

background-position: 50% 50%;

5. background-size

Specifies the size of the background image.

Example:

background-size: cover; /* covers entire element */

background-size: contain; /* fits image inside element */

background-size: 100px 200px;

CSS Gradients

Gradients are a way to create smooth transitions between two or more colors.

1. Linear Gradients

Syntax:

background: linear-gradient(direction, color1, color2, ...);

- **Direction examples:**
  - to right (left to right)
  - to bottom (top to bottom)
  - 45deg (angle)

Example:

background: linear-gradient(to right, red, yellow);

You can add multiple color stops:

Syntax:

background: linear-gradient(to right, red, orange, yellow, green, blue, indigo, violet);

2. Radial Gradients

The color transition radiates from a central point.

Syntax:

background: radial-gradient(shape size at position, color1, color2, ...);

Basic example:

background: radial-gradient(circle, red, yellow, green);

With position and size:

background: radial-gradient(circle at center, red, yellow, green);

Extra: Shorthand Property

You can use the background shorthand:

Syntax:

background: #fff url("image.jpg") no-repeat center center / cover;

Full Example: Colors and Backgrounds

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <title>Techpile Technology</title>
 <style>
  /* Apply styles to the full page */
  body {
   /* Background color (fallback) */
   background-color: #f0f0f0;
   /* Background image */
```

```
    background-image:
     linear-gradient(rgba(255, 0, 0, 0.3), rgba(0, 0, 255, 0.3)), /* RGBA linear gradient overlay */
     url("https://via.placeholder.com/1200x800"); /* background image */
    /* Repeat and position */
    background-repeat: no-repeat;
    background-position: center center;
    background-size: cover;
    /* Font and color using HSL */
    font-family: Arial, sans-serif;
    color: hsl(210, 100%, 20%);
    text-align: center;
    padding: 50px;
  }
  .content {
    background: radial-gradient(circle at center, #ffffffcc, #ffffff00);
    border-radius: 20px;
    padding: 30px;
    max-width: 600px;
    margin: auto;
    box-shadow: 0 8px 20px rgba(0, 0, 0, 0.3);
  }

  h1 {
    color: #ff4500; /* HEX color */
```

```
  }
  p {
    color: rgb(50, 50, 50); /* RGB color */
  }
  .transparent-note {
    color: rgba(0, 0, 0, 0.6); /* RGBA text */
    font-style: italic;
  }
 </style>
</head>
<body>
 <div class="content">
  <h1>Code Helper</h1>
  <p>Summer Tranning.</p>
  <p class="transparent-note"> Apprenticeship Program </p>
 </div>
</body>
</html>
```

Module 4: Fonts and Text Styling (CSS)

Fonts and text styling are crucial for making web content **readable**, **attractive**, and **professional**. CSS provides various properties to control the appearance of text on a webpage.

1. CSS Text and Font Properties

font-family

- **What it does**: Specifies the font to be used for text.
- **How it works**: You provide a list of fonts. If the browser doesn't have the first font, it tries the next.

  Syntax:

  font-family: "Arial", "Helvetica", sans-serif;

- **Best Practice**: Always include a **generic family** (sans-serif, serif, monospace) as fallback.

font-size

- **What it does**: Sets the size of the font.
- **Units**:
  - px – pixels (fixed size)
  - em – relative to the parent
  - rem – relative to the root element
  - % – relative to parent
- **Syntax**:

  font-size: 16px;

  font-size: 1.2em;

font-style

- **What it does**: Specifies the style of the text (such as italic or oblique).
- **Values**:
    - normal – default style
    - italic – italic text
    - oblique – slanted text
- **Syntax**:

  font-style: italic;

font-weight

- **What it does**: Defines how bold or light the text is.
- **Values**:
    - normal, bold, lighter, bolder
    - Numeric values: 100 (thin) to 900 (extra bold)
- **Syntax**:

  font-weight: bold;

  font-weight: 700;

text-align

- **What it does**: Aligns text **horizontally** within its container.
- **Values**:
    - left (default)
    - center
    - right
    - justify (aligns both left and right edges)

- **Syntax**:

  text-align: center;

text-transform

- **What it does**: Controls the capitalization of text.
- **Values**:
    - uppercase – converts text to uppercase
    - lowercase – converts text to lowercase
    - capitalize – capitalizes the first letter of each word
- **Syntax**:

text-transform: capitalize;

text-decoration

- **What it does**: Adds decorations to the text.
- **Values**:
    - none
    - underline
    - overline
    - line-through
- **Syntax**:

text-decoration: underline;

line-height

- **What it does**: Sets the amount of space between lines of text (line spacing).
- **Syntax**:

line-height: 1.5;

letter-spacing

- **What it does**: Controls the spacing **between letters**.
- **Syntax**:

letter-spacing: 2px;

word-spacing

- **What it does**: Controls the spacing **between words**.

- **Syntax**:

  word-spacing: 5px;

Google Fonts Integration

What is Google Fonts?

Google Fonts is a free web service by Google that offers 1000+ stylish and professional fonts which you can use directly in your website **without downloading** them.

Why Use Google Fonts?

- Access to modern, professional fonts
- Easy to implement
- Improves design without performance issues
- Fast CDN loading
- Free and open-source
- Supports multiple languages

How to Use Google Fonts

Step 1: Include the font in HTML

Use the <link> tag inside the <head> of your HTML file:

Example:

```
<link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap" rel="stylesheet">
```

Step 2: Apply the font in CSS

Syntax:

```
body {
  font-family: 'Roboto', sans-serif;
}
```

Example:

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
 <meta charset="UTF-8">
 <title>Techpile Technology</title>
 <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;700&display=swap"
rel="stylesheet">
 <style>
  body {
   font-family: 'Poppins', sans-serif;
  }
  h1 {
   font-weight: 700;
  }
  p {
   font-weight: 400;
  }
 </style>
</head>
<body>
 <h1>This is a Bold Heading</h1>
 <p>This is a regular paragraph text.</p>
</body>
</html>
```

Features of Google Fonts

- 100% Free to use
- 1000+ font families available
- Hosted on fast, reliable Google servers
- Easy to integrate with any HTML/CSS project
- Customizable weights and styles (light, bold, italic, etc.)
- Open-source, no licensing issues

Module 5: CSS Box Model

What is the Box Model?
Every HTML element is considered a **rectangular box** by the browser. The CSS Box Model describes the design and layout of these boxes, defining how elements take up space on the page.

The box model consists of **4 main parts** (from outside to inside):

1. **Margin** – The outermost space around the element.
2. **Border** – The edge or frame surrounding the element.
3. **Padding** – Space between the content and the border.
4. **Content** – The actual content (text, image, etc.).

1.Margin

- **What it is:** The space **outside** the border, separating elements from others.
- **Purpose:** Controls the distance between elements.
- **Values:** Length units (px, em, %), or auto.
- **Syntax:**

  margin: 20px; /* all sides */

  margin-top: 10px; /* individual side */

  margin: 10px 20px; /* vertical | horizontal */

2.Border

- **What it is:** The line that surrounds the padding and content.
- **Purpose:** Defines an element's boundary, can be styled with color, width, and style.
- **Syntax:**

  border: 2px solid black;

  border-top: 5px dashed red;

- **Properties:**
  - border-width

- border-style (solid, dashed, dotted, none)
- border-color

3.Padding

- **What it is:** The space **inside** the border, between border and content.
- **Purpose:** Adds breathing room inside the element.
- **Syntax:**

  padding: 15px;

  padding-left: 10px;

  padding: 10px 20px; /* vertical | horizontal */

4.Width and Height

- **What they are:** Dimensions of the **content box** by default.
- **Default behavior:** width and height define the size of the **content only**, excluding padding, border, and margin.
- **Syntax:**

  width: 300px;

  height: 150px;

- **Note:** Total space used = width + padding + border + margin.

5.box-sizing: border-box

What is box-sizing?

box-sizing controls how the browser calculates the **total width and height** of an element.

- **Default value:** content-box
  Width and height apply only to the **content** area. Padding and border add extra space outside this.
- **With border-box:**
  Width and height include **content + padding + border** — so the element fits exactly into the specified width and height.

Why use box-sizing: border-box?

It makes sizing elements easier and more intuitive because the total size includes padding and border — so you don't accidentally make elements bigger than expected.

Syntax:

box-sizing: border-box;

Example of difference:

```
/* Default (content-box) */
div {
  width: 300px;
  padding: 20px;
  border: 5px solid black;
  box-sizing: content-box; /* default */
}
/* Using border-box */
div {
  width: 300px;
  padding: 20px;
  border: 5px solid black;
  box-sizing: border-box;
}
```

- In **content-box**, total width = 300 + 20*2 (padding) + 5*2 (border) = 350px
- In **border-box**, total width = 300px (including padding and border)

Practical Tip:

Most developers use this CSS globally for easier layouts:

```
* {
```

```
  box-sizing: border-box;
}
```

Summary Table

Property	What it controls	Notes
margin	Space outside the border	Separates element from others
border	Line around padding+content	Style, thickness, color
padding	Space inside border	Between content and border
width/height	Content box size	Default excludes padding and border
box-sizing	How width/height is calculated	border-box includes padding & border

Module 6: Display and Positioning in CSS

1.Display

The display property controls how an element is displayed in the document flow.

Common values:

block

- Element takes up full width available.
- Starts on a new line.
- Examples: , <p>, <h1> by default.

Example:

display: block;

inline

- Element takes width of its content only.
- Does NOT start on a new line; flows inline with other elements.
- Examples: , <a>.

Example:

display: inline;

inline-block

- Behaves like inline (does not start new line) but respects width & height.
- Combines benefits of block and inline.

Example:

display: inline-block;

none

- Element is not displayed at all (removed from layout).
- Useful for hiding elements without deleting them from HTML.

Example:

```
display: none;
```

2.Visibility

Controls whether an element is visible or hidden, **but it still takes up space** if hidden.

- **visible** — element is shown (default)
- **hidden** — element is invisible but space is preserved
- **collapse** — similar to hidden but used mainly with tables to remove rows/columns

Example:

```
visibility: hidden;
```

Difference from display:none:

display:none removes element from the layout, visibility:hidden hides it but keeps the space reserved.

3.position

Controls how an element is positioned in the document.

Values:

- **static** (default)
  - Elements follow normal document flow.
  - Top, left, right, bottom properties do **not** apply.

Example:

```
position: static;
```

- **relative**
  - Positioned relative to its normal position.
  - You can move it using top, right, bottom, left.
  - Space is reserved in original position (element still takes original spot).

Example:

position: relative;

top: 10px; /* moves element 10px down from normal */

left: 20px; /* moves 20px right */

- **absolute**
    - Positioned relative to nearest **positioned ancestor** (not static).
    - Removed from normal flow (doesn't take up space).
    - Moves based on top, right, bottom, left relative to ancestor.

Example:

position: absolute;

top: 0;

left: 0;

- **fixed**
    - Positioned relative to the **viewport** (browser window).
    - Stays fixed when scrolling.
    - Removed from document flow.

Example:

position: fixed;

top: 10px;

right: 10px;

- **sticky**
    - Acts like relative until a defined scroll position is reached, then behaves like fixed.
    - Useful for headers that stick on top after scrolling.

Example:

position: sticky;

top: 0;

4.top, right, bottom, left

- These properties move the element **depending on the position value** (relative, absolute, fixed, or sticky).
- They specify offset from the respective edges.

Property	**Meaning**
top	Distance from the top edge
right	Distance from the right edge
bottom	Distance from the bottom edge
left	Distance from the left edge

Example:

position: absolute;

top: 20px;

left: 50px;

This places the element 20px from the top and 50px from the left of its positioned ancestor.

5. z-index

- Controls **stacking order** of overlapping positioned elements (position other than static).
- Elements with higher z-index appear on top.
- Only works on elements with position relative, absolute, fixed, or sticky.
- Default z-index is auto (elements stack in source order).

Example:

position: relative;

z-index: 10;

- z-index can be positive, negative, or zero.
- Higher values stack above lower values.

Summary Table

Property	**What it does**	**Notes**
display	Controls how element behaves in layout	block, inline, inline-block, none
visibility	Hides element visually but keeps space	hidden vs display:none difference
position	Controls element positioning	static, relative, absolute, fixed, sticky
top/right/bottom/left	Offsets for positioned elements	Works only if position is not static
z-index	Controls layering order (stacking)	Works only for positioned elements

Module 7: Flexbox (Flexible Box Layout)

Flexbox is a powerful layout model in CSS used to arrange elements in **rows or columns**, align items easily, and handle spacing—even when screen sizes change.

1. Flex Container and Flex Items

- A **Flex Container** is the parent element where Flexbox is applied.
- **Flex Items** are the direct children of that container.

Example:

```
<style>
.container {
  display: flex;
}
</style>
<div class="container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
</div>
```

2. display: flex

- Turns the element into a **flex container**.
- Its direct children become **flex items**.
- Default direction is **row** (horizontal layout).

Example:

```
.container {
  display: flex;
}
```

3. flex-direction

- Controls **direction** of items inside the container.

Value	Description
row	Horizontal (default)
row-reverse	Horizontal reverse
column	Vertical
column-reverse	Vertical reverse

Example:

```
.container {

 flex-direction: column;

}
```

4. justify-content

- Aligns items **horizontally** (main axis).

Value	Effect
flex-start	Align to the start (left)
flex-end	Align to the end (right)
center	Center items
space-between	Space between items only
space-around	Space on both sides of items
space-evenly	Equal space around all items

Example:

```
.container {
 justify-content: space-between;
}
```

5. align-items

- Aligns items **vertically** (cross axis).
- Applies to **all flex items**.

Value	Effect
stretch	Default; stretch to fill container
flex-start	Align items to top/start
flex-end	Align items to bottom/end
center	Center items vertically
baseline	Align text baselines

Example:

```
.container {
 align-items: center;
}
```

6. align-self

- Overrides align-items **for a single item**.
- Values are same as align-items.

Example:

```
.item:nth-child(2) {
 align-self: flex-end;  }
```

7. flex-wrap

- Controls whether items **wrap to the next line** when there's no space.

Value	Description
nowrap	All items stay on one line (default)
wrap	Items wrap to the next line
wrap-reverse	Wrap in reverse order

Example:

```
.container {
  flex-wrap: wrap;
}
```

8. gap

- Adds **space between flex items**.
- Works in both row and column directions.

Example:

```
.container {
  gap: 20px;
}
```

This is **much cleaner** than using margin on items for spacing.

Flexbox Full Example

```
<style>
  .container {
    display: flex;
    flex-direction: row;
```

```
    flex-wrap: wrap;
    justify-content: space-between;
    align-items: center;
    gap: 20px;
    background: #f0f0f0;
    padding: 20px;
  }
  .item {
    background: #4CAF50;
    color: white;
    padding: 20px;
    width: 100px;
    text-align: center;
  }
</style>
<div class="container">
  <div class="item">HTML</div>
  <div class="item">CSS</div>
  <div class="item">JavaScript</div>
  <div class="item">BootStrap</div>
</div>
```

Summary Table

Property	Use
display: flex	Defines flex container
flex-direction	Sets direction of flex items
justify-content	Aligns items horizontally
align-items	Aligns items vertically
align-self	Aligns a single item on cross axis
flex-wrap	Allows items to wrap to the next line
gap	Adds spacing between items

Module 8: CSS Grid Layout

CSS Grid is a two-dimensional layout system for the web — it can handle **rows and columns at the same time**, making it powerful for creating complex layouts.

1. display: grid

- Turns an element into a **grid container**.
- Its direct children become **grid items**.

Example:

```
.container {
  display: grid;
}
```

2. grid-template-columns and grid-template-rows

These properties define the **structure of the grid** (how many columns and rows, and their sizes).

grid-template-columns

Defines the number and width of columns.

Example:

```
.container {
  display: grid;
  grid-template-columns: 100px 200px auto;
}
```

- You can also use **repeat()** to simplify:

Example:

```
grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
```

grid-template-rows

Defines the height of each row.

Example:

```
grid-template-rows: 100px 150px auto;
```

3. gap

Adds space **between** grid items. Can be used in one or both directions.

Example:

```
.container {
  gap: 20px; /* row and column gap */
}
.container {
  row-gap: 10px;
  column-gap: 20px;
}
```

4. grid-column, grid-row

Used on **individual grid items** to define how much space they span.

Example:

```
.item1 {
  grid-column: 1 / 3; /* spans from column 1 to 3 */
  grid-row: 1 / 2;     /* row 1 only */
}
```

You can also use span:

Example:

```
grid-column: span 2; /* spans 2 columns */
```

5. grid-area

Used to place an item in a specific area of the grid by **naming the areas**.

Step 1: Define area names in container:

Example:

```
.container {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  grid-template-columns: 1fr 3fr;
  grid-template-rows: auto 1fr auto;
}
```

Step 2: Assign grid items to those areas:

Example:

```
.header  { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; }
.footer  { grid-area: footer; }
```

Example of a Simple Grid Layout

```
<style>
  .container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    grid-template-rows: auto;
    gap: 20px;
    padding: 20px;
```

```
 }

 .item {
  background: #4CAF50;
  color: white;
  padding: 20px;
  text-align: center;
 }
 .item:nth-child(1) {
  grid-column: span 2;
 }
</style>
<div class="container">
 <div class="item">1 (span 2 columns)</div>
 <div class="item">2</div>
 <div class="item">3</div>
 <div class="item">4</div>
 <div class="item">5</div>
</div>
```

Combining Grid and Flexbox

- CSS Grid is great for **overall layout** (page structure).
- Flexbox is perfect for **content alignment inside components** (like navbars, cards, buttons).

Example: Grid layout + Flexbox inside items

```
<style>
```

```
.container {
  display: grid;
  grid-template-columns: 1fr 2fr;
  gap: 20px;
}
.sidebar, .main {
  background: #ddd;
  padding: 20px;
}
.nav {
  display: flex;
  justify-content: space-between;
  background: #4CAF50;
  padding: 10px;
  color: white;
}
</style>
<div class="container">
 <div class="sidebar">
  <div class="nav">
   <span>Menu</span>
   <span>Profile</span>
  </div>
  Sidebar content
 </div>
```

```
 <div class="main">Main content here</div>

</div>
```

- The **container** uses **Grid** for layout.
- The **.nav** inside uses **Flexbox** for spacing between menu items.

Summary Table

Property	**What it does**
display: grid	Defines a grid container
grid-template-columns	Defines column count and size
grid-template-rows	Defines row height
gap	Sets space between items
grid-column	Controls column span of an item
grid-row	Controls row span of an item
grid-area	Places items in named areas
Combine with Flexbox	Use Grid for layout, Flexbox for alignment inside

Module 9: Responsive Design in CSS

Responsive design ensures that your website **adapts to different screen sizes** (mobile, tablet, desktop) for better usability and appearance.

1. Media Queries

Media Queries allow you to apply **CSS rules based on device properties**, like screen width, height, orientation, etc.

Syntax:

```
@media (condition) {
  /* CSS rules */
}
```

Example:

```
/* Apply only on screens 600px or less */
@media (max-width: 600px) {
  body {
    background-color: lightblue;
  }
}
```

Common Conditions:

- (max-width: 768px) → Target mobile & tablets
- (min-width: 769px) → Target larger screens
- (orientation: portrait) → Target vertical screens

2. max-width & min-width

These are often used inside media queries:

Property	Meaning
max-width	Applies styles **up to** a certain width
min-width	Applies styles **starting from** a certain width

Example:

```
/* Tablet and smaller */
@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
}
/* Desktop and larger */
@media (min-width: 769px) {
  .container {
    flex-direction: row;
  }
}
```

3. Responsive Units

Responsive units help layout and text **scale based on screen size**.

% (percentage)

- Based on **parent element** size.
- Used for widths, paddings, margins.

Example:

width: 50%; /* 50% of parent */

vw, vh (viewport width/height)

- Based on **viewport** (screen).
- 1vw = 1% of screen width
- 1vh = 1% of screen height

Example:

width: 100vw; /* full screen width */

height: 100vh; /* full screen height */

em, rem

- Used for **typography and spacing**
- em = relative to **parent font size**
- rem = relative to **root font size** (usually 16px)

Example:

font-size: 2em; /* 2x parent font size */

font-size: 1.5rem; /* 1.5x root font size */

4. Mobile-First Approach

Mobile-first means:

- Design for **small screens first**
- Use min-width media queries to add styles for larger screens

Benefits:

Better performance on mobile
Easier to scale up than scale down
Recommended practice by most developers

Example:

/* Base style: for mobile */

.container {

```
  flex-direction: column;
}
/* Add desktop layout */
@media (min-width: 768px) {
  .container {
    flex-direction: row;
  }
}
```

5. Responsive Typography

Typography that **scales with screen size** improves readability.

Techniques:

1. Using relative units (em/rem)

```
body {
  font-size: 1rem; /* base font size */
}
h1 {
  font-size: 2rem; /* 2x base */
}
```

2. Using clamp() function

Modern way to set scalable font size with min and max limits.

```
h1 {
  font-size: clamp(1.5rem, 2.5vw, 3rem);
}
```

This means:

- Minimum size = 1.5rem

- Preferred = 2.5% of viewport width
- Maximum = 3rem

Responsive Design Example

Example:

```
<style>
 body {
  font-family: sans-serif;
  font-size: 1rem;
 }
 .container {
  display: flex;
  flex-direction: column;
  padding: 1rem;
 }
 @media (min-width: 768px) {
  .container {
   flex-direction: row;
   justify-content: space-between;
  }
 }
 h1 {
  font-size: clamp(1.5rem, 5vw, 3rem);
 }
</style>
<div class="container">
```

```
  <h1>Responsive Design</h1>
  <p>This layout changes based on screen size.</p>
</div>
```

Summary Table

Concept	Use
@media	Apply styles based on device conditions
max-width/min-width	Target screen size ranges
%	Relative to parent
vw/vh	Relative to screen size
em/rem	Relative to font sizes
clamp()	Responsive font scaling
Mobile-First	Default to mobile, enhance for larger screens

Module 10: CSS Transitions and Animations

CSS lets you create **smooth visual effects** without JavaScript using:

- **Transitions** — changes happen gradually
- **Animations** — complex multi-step movements using @keyframes

1. CSS Transitions

Transitions let you animate changes in **CSS property values** when something (like hover or focus) happens.

Syntax:

```
selector {
  transition-property: property-name;
  transition-duration: time;
  transition-timing-function: type;
}
```

transition-property

Specifies which property you want to animate.

Example:

```
transition-property: background-color;
```

You can also use all to apply transition to all animatable properties.

transition-duration

How long the transition takes (in seconds or milliseconds).

Example:

```
transition-duration: 0.5s; /* 0.5 seconds */
```

transition-timing-function

Defines the **speed curve** of the transition.

Value	Description
ease	Starts slow, speeds up, then slows down
linear	Same speed from start to finish
ease-in	Starts slow, then fast
ease-out	Starts fast, then slows
ease-in-out	Slow → Fast → Slow

Example: Hover effect using transition

```
<style>
.button {
  background-color: blue;
  color: white;
  padding: 10px 20px;
  transition: background-color 0.3s ease;
}
.button:hover {
  background-color: red;
}
</style>
<button class="button">Hover Me</button>
```

2. CSS Animations using @keyframes

Animations let you **define multiple stages** of changes using @keyframes.

Syntax:

```
@keyframes animationName {
  0%   { property: value; }
  50%  { property: value; }
  100% { property: value; }
}
```

You then attach the animation to a selector.

animation-name

Specifies the name of the @keyframes to use.

animation-duration

Sets **how long** the animation should run.

Example:

```
animation-duration: 2s;
```

animation-delay

Delays the **start** of the animation.

Example:

```
animation-delay: 1s; /* wait 1 second before starting */
```

Example: Animate a box from left to right

```
<style>
@keyframes slideRight {
  from {
    transform: translateX(0);
  }
```

```
 to {
   transform: translateX(300px);
 }
}
.box {
 width: 100px;
 height: 100px;
 background-color: green;
 animation-name: slideRight;
 animation-duration: 2s;
 animation-delay: 0s;
}
</style>
<div class="box"></div>
```

Combine Animation Properties

You can write animation shorthand like this:

Example:

```
animation: slideRight 2s ease-in-out 1s infinite;
```

- animation-name: slideRight
- animation-duration: 2s
- animation-timing-function: ease-in-out
- animation-delay: 1s
- animation-iteration-count: infinite

Example: Bounce animation

```
<style>
@keyframes bounce {
  0%   { transform: translateY(0); }
  50%  { transform: translateY(-50px); }
  100% { transform: translateY(0); }
}
.ball {
  width: 50px;
  height: 50px;
  background-color: red;
  border-radius: 50%;
  animation: bounce 1s ease-in-out infinite;
}
</style>

<div class="ball"></div>
```

Summary Table

Property	Use
transition-property	Which property to animate on change
transition-duration	How long the transition takes
transition-timing-function	How speed varies during transition
@keyframes	Defines key stages of animation
animation-name	Links element to keyframes
animation-duration	Total animation time
animation-delay	Delay before animation starts

Module 11: Advanced Topics in CSS

1. CSS Variables (Custom Properties)

CSS variables allow you to **reuse values** (like colors, font sizes) throughout your CSS with easy maintenance.

Syntax:

```
:root {
  --primary-color: #4CAF50;
  --padding: 1rem;
}
button {
  background-color: var(--primary-color);
  padding: var(--padding);
}
```

Explanation:

- :root is like the global scope for CSS.
- --variable-name defines a variable.
- var(--variable-name) uses the variable.

Benefits:

- Easier to manage design system (colors, spacing, fonts).
- Supports **theming**, like light/dark mode.

2. Custom Scrollbars (WebKit Browsers)

You can style scrollbars using vendor-prefixed pseudo-elements (mostly for Chrome, Safari, Edge).

Example:

```
/* Total scrollbar */
::-webkit-scrollbar {
```

```
 width: 10px;
}
/* Scrollbar track */
::-webkit-scrollbar-track {
 background: #f1f1f1;
}
/* Scrollbar thumb */
::-webkit-scrollbar-thumb {
 background: #888;
 border-radius: 10px;
}
::-webkit-scrollbar-thumb:hover {
 background: #555;
}
```

Note:

- Not fully supported in Firefox (use scrollbar-color and scrollbar-width for Firefox).

3. CSS Filters

Filters apply **visual effects** like blur, brightness, or grayscale to images and elements.

Syntax:

```
img {
 filter: blur(5px);
}
```

Available Filters:

Filter	Description	Example
blur(px)	Blurs the image	blur(4px)
brightness(%)	Makes image lighter/darker	brightness(150%)
grayscale(%)	Turns image black & white	grayscale(100%)
contrast(%)	Adjusts contrast	contrast(120%)
sepia(%)	Old-photo effect	sepia(80%)

Example:

```
img:hover {
  filter: grayscale(100%) brightness(1.2);
}
```

4. Responsive Images with object-fit

object-fit defines **how media (like images or videos) fit inside their containers**.

Syntax:

```
img {
  width: 100%;
  height: 300px;
  object-fit: cover;  }
```

Values:

Value	Description
fill	Default. Stretches image to fill container (may distort)
contain	Fits the image inside without cropping

Value	Description
cover	Fills the container, cropping if needed
none	No resizing

scale-down Like none or contain, whichever is smaller

Best for:

- Making **responsive banners**, thumbnails, or cards look good without distortion.

5. Dark Mode Using Media Queries

You can detect user's **system theme preference** and apply styles accordingly.

Syntax:

```
@media (prefers-color-scheme: dark) {
  body {
    background-color: #121212;
    color: #ffffff;
  }
}
@media (prefers-color-scheme: light) {
  body {
    background-color: #ffffff;
    color: #000000;
  }
}
```

How it works:

- If user has enabled **dark mode** on their device, the dark styles are applied.
- Great for **accessibility** and modern UI preferences.

Summary Table

Feature	Use
--css-variable	Reuse colors/values in CSS
::-webkit-scrollbar	Customize scrollbar appearance
filter:	Add effects like blur, grayscale, brightness
object-fit:	Control how images resize inside their containers
prefers-color-scheme	Apply styles based on system light/dark mode preference